

StreamFlix Database Schema Creation and CSV Data Import Documentation

Objective

The objective of this task was to create a SQL Server database schema for the StreamFlix project and import data from CSV files into the created tables.

The database was designed to store information about movies, users, and ratings. The ratings table links users and movies together using foreign keys.

Tools Used

- SQL Server Management Studio (SSMS)
 - SQL Server Import and Export Wizard
 - CSV files:
 - Movies.csv / Movies_Cleaned.csv
 - Users.csv
 - Ratings_Dataset.csv
-

Database Name

DA07_Movies_Db

Tables Created

Three main tables were created:

1. Movies
 2. Users
 3. Ratings
-

1. Movies Table

The Movies table stores movie-related information.

```
CREATE TABLE Movies (  
    movie_id INT PRIMARY KEY,  
    title NVARCHAR(1000),
```

```
    release_year INT,  
    genre NVARCHAR(500),  
    language VARCHAR(50),  
    country VARCHAR(50),  
    total_views INT  
);
```

Key Details

- movie_id is the primary key.
 - title was changed to NVARCHAR(1000) to avoid truncation errors during CSV import.
 - genre was changed to NVARCHAR(500) because some movies contain multiple genres.
-

2. Users Table

The Users table stores user demographic and subscription information.

```
CREATE TABLE Users (  
    user_id VARCHAR(20) PRIMARY KEY,  
    age INT,  
    gender CHAR(1),  
    country VARCHAR(50),  
    subscription_status VARCHAR(50),  
    total_watch_time INT,  
    device VARCHAR(50)  
);
```

Key Details

- user_id is the primary key.
 - This table includes user age, gender, country, subscription status, total watch time, and device type.
-

3. Ratings Table

The Ratings table stores rating information and connects users with movies.

```
CREATE TABLE Ratings (  
    rating_id INT PRIMARY KEY,  
    user_id VARCHAR(20),  
    movie_id INT,  
    rating DECIMAL(2,1),  
    rating_timestamp DATETIME,  
  
    CONSTRAINT FK_Ratings_Users
```

```
        FOREIGN KEY (user_id)
        REFERENCES Users(user_id),

    CONSTRAINT FK_Ratings_Movies
        FOREIGN KEY (movie_id)
        REFERENCES Movies(movie_id)
);
```

Key Details

- rating_id is the primary key.
 - user_id is a foreign key linked to the Users table.
 - movie_id is a foreign key linked to the Movies table.
 - This table creates the relationship between users and movies.
-

Data Import Process

The CSV files were imported into the existing SQL Server tables using the SQL Server Import and Export Wizard.

Import Steps

1. Opened SQL Server Management Studio.
2. Connected to the SQL Server database.
3. Right-clicked the database DA07_Movies_Db.
4. Selected:

Tasks → Import Data

5. Selected the CSV file as the source.
 6. Selected SQL Server as the destination.
 7. Mapped the CSV columns to the existing database table columns.
 8. Chose append/import into the existing destination table.
 9. Completed the import wizard.
 10. Verified the imported data using SQL queries.
-

Import Order

The data was imported in the following order:

1. Movies table
2. Users table
3. Ratings table

This order was important because the Ratings table contains foreign keys referencing the Movies and Users tables.

Issues Faced and Fixes

Issue 1: Title Truncation Error

During movie import, an error occurred because the Title column contained long text or special characters.

Fix

The Movies table column was updated:

```
ALTER TABLE Movies
ALTER COLUMN title NVARCHAR(1000);
```

```
ALTER TABLE Movies
ALTER COLUMN genre NVARCHAR(500);
```

In the Import Wizard, the Title column width was also increased to avoid truncation.

Issue 2: Foreign Key Error in Ratings Table

A foreign key error occurred when importing ratings because some user_id values in Ratings did not match the Users table.

Fix

The Users table was checked and imported before importing the Ratings table.

```
SELECT COUNT(*) AS TotalUsers
FROM Users;
```

Validation Queries

Check Imported Data

```
SELECT TOP 10 * FROM Movies;
SELECT TOP 10 * FROM Users;
SELECT TOP 10 * FROM Ratings;
```

Check Row Counts

```
SELECT COUNT(*) AS TotalMovies FROM Movies;
SELECT COUNT(*) AS TotalUsers FROM Users;
SELECT COUNT(*) AS TotalRatings FROM Ratings;
```

Test Table Relationships

```
SELECT
    r.rating_id,
    u.user_id,
    u.country,
    m.title,
    r.rating,
    r.rating_timestamp
FROM Ratings r
JOIN Users u
    ON r.user_id = u.user_id
JOIN Movies m
    ON r.movie_id = m.movie_id;
```

Schema Relationship Summary

- One user can give many ratings.
- One movie can receive many ratings.
- The Ratings table acts as a bridge between Users and Movies.

Relationship:

Users (1) → Ratings (Many)

Movies (1) → Ratings (Many)

Conclusion

The database schema was successfully created in SQL Server. The Movies, Users, and Ratings tables were designed with primary keys and foreign keys to support relational analysis. CSV files were imported into the existing tables, and validation queries were used to confirm that the data was loaded and relationships were working correctly.